PROJETO E ANÁLISE DE ALGORITMOS · PROJETO FINAL

Batalha dos Algoritmos de Ordenação

Análise teórica e experimental de Bubble, Insertion, Selection, Merge e Quick Sort

Disciplina	Projeto e Análise de Algoritmos
Professor	Alexandre Barbosa de Souza
Integrantes	Carlos Eduardo Pisa Meireles Deivid Ramos dos Reis Silva Diana Carina Frederich Briere Eduardo Augusto Rosario Maciel Ruan Victor de Santana Batista
Entrega	22/11/2026

1. Introdução

1.1 Contexto

Ordenar uma coleção é uma das operações mais básicas da computação e serve de laboratório clássico para a análise de algoritmos: problemas simples, soluções com custos assintóticos muito diferentes e fácil verificação do resultado. Neste projeto, cinco algoritmos foram implementados em Java e comparados experimentalmente, a fim de confrontar o que a teoria prevê (notação assintótica) com o que realmente acontece em uma máquina real (cache, compilador JIT, previsão de desvios, coleta de memória).

1.2 Objetivos

- Implementar Bubble, Insertion, Selection, Merge e Quick Sort seguindo a interface `SortingAlgorithm` (`sort`, `getComparisons`, `getSwaps`, `getExecutionTime`, `isSorted`).
- Medir comparações, trocas (ou movimentos), tempo de execução e memória auxiliar para entradas aleatórias, ordenadas e invertidas, em vários tamanhos.
- Apresentar os resultados em gráficos log-log, compará-los com a teoria e explicar as discrepâncias.
- Discutir os *trade-offs* e indicar qual algoritmo é adequado para cada situação.

1.3 Descrição dos algoritmos

Bubble Sort. Percorre o vetor repetidamente comparando pares adjacentes e trocando-os quando estão fora de ordem; a cada passada o maior elemento restante "sobe" para o fim. A versão implementada interrompe quando uma passada não realiza trocas, o que lhe dá custo linear em vetores já ordenados. É estável e in-place.

Insertion Sort. Mantém um prefixo ordenado e insere cada novo elemento na posição correta, deslocando para a direita os elementos maiores. As "trocas" contadas são esses deslocamentos (movimentos). É estável, in-place e muito eficiente em vetores quase ordenados.

Selection Sort. A cada passo seleciona o menor elemento do trecho não ordenado e o coloca na posição atual. Faz sempre o mesmo número de comparações, independentemente da entrada, mas no máximo $n-1$ trocas (só conta troca quando o índice do menor é diferente do atual). É in-place e não estável.

Merge Sort. Divide e conquista: ordena as metades e as intercala. A implementação é iterativa (*bottom-up*), toda dentro de `sort`: intercala blocos de tamanho 1, 2, 4, … usando um vetor auxiliar alocado uma única vez. É estável, tem custo $\Theta(n \log n)$ em todos os casos e usa $O(n)$ de memória extra. As "trocas" contadas são as escritas de volta no vetor original.

Quick Sort. Escolhe um pivô, particiona o vetor em menores e maiores e repete nos dois trechos. A implementação usa pivô aleatório (`Random(42)`), partição com dois

ponteiros e uma pilha explícita de 128 inteiros no lugar de recursão, processando sempre o trecho menor primeiro, de modo que a pilha cresce no máximo O(log n). É in-place e não estável.

2. Fundamentação Teórica

Notação: **O(f)** é um limite superior (o algoritmo não é pior que f, a menos de constante); **Ω(f)** é um limite inferior (não é melhor que f); **Θ(f)** é limite justo, quando O e Ω coincidem. Quando o melhor e o pior caso diferem, não há um único Θ para o algoritmo todo e informamos Θ por caso.

Algoritmo	Melhor caso	Caso médio	Pior caso	Ω	O	Memória extra	Estável
Bubble (com parada antecipada)	Θ(n)	Θ(n²)	Θ(n²)	Ω(n)	O(n²)	O(1)	sim
Insertion	Θ(n)	Θ(n²)	Θ(n²)	Ω(n)	O(n²)	O(1)	sim
Selection	Θ(n²)	Θ(n²)	Θ(n²)	Ω(n²)	O(n²)	O(1)	não
Merge	Θ(n log n)	Θ(n log n)	Θ(n log n)	Ω(n log n)	O(n log n)	O(n)	sim
Quick (pivô aleatório)	Θ(n log n)	Θ(n log n)	Θ(n²)	Ω(n log n)	O(n²)	O(log n)	não

2.1 Justificativas

Bubble. Na passada i são feitas n−i comparações, o que soma $n(n-1)/2 = \Theta(n^2)$ quando há trocas em quase todas as passadas (vetor aleatório ou invertido). Com a parada antecipada, um vetor ordenado exige uma única passada de n−1 comparações: Ω(n). O número de trocas é igual ao de inversões do vetor: 0 no ordenado, n(n−1)/4 em média e n(n−1)/2 no invertido.

Insertion. O elemento da posição i desloca tantos elementos quantos forem maiores que ele à esquerda (as inversões). Ordenado: 1 comparação por elemento, logo n−1 comparações e nenhum movimento, Θ(n). Invertido: i comparações e i movimentos na posição i, somando $n(n-1)/2$, $\Theta(n^2)$. Em média, cada par está invertido com probabilidade 1/2, portanto cerca de n(n−1)/4 movimentos, ainda $\Theta(n^2)$.

Selection. O laço interno sempre percorre todo o trecho restante, independentemente dos valores: $\Sigma(n-1-i) = n(n-1)/2$ comparações em qualquer entrada, logo $\Theta(n^2)$ em todos os casos. Trocas: no máximo n−1, pois há no máximo uma por passo; em média cerca de $n-H_n$ (H_n é o n-ésimo número harmônico) e, no vetor invertido, n/2.

Merge. A recorrência $T(n) = 2T(n/2) + \Theta(n)$ resolve para $\Theta(n \log n)$ pelo Teorema Mestre (caso 2). Na versão iterativa há $\lceil \log_2 n \rceil$ níveis e cada nível percorre os n elementos uma vez. Cada intercalação de dois blocos custa entre $\min(|A|,|B|)$ e $|A|+|B|-1$ comparações; por isso o número de comparações varia entre aproximadamente $(n \log_2 n)/2$ (melhor caso) e $n \log_2 n - n + 1$ (pior caso), mas as escritas de volta são sempre n por nível, e o custo assintótico não muda. O vetor auxiliar de n posições dá a memória O(n).

Quick. Cada partição custa Θ(n) no trecho. Se o pivô divide o trecho de forma equilibrada, $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$. Se o pivô for sempre o menor ou o

maior elemento, $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Com pivô aleatório, o número esperado de comparações é aproximadamente $2n \ln n \approx 1{,}39\, n \log_2 n$, e o pior caso ocorre com probabilidade desprezível. Como o trecho menor é processado primeiro, a pilha de ranges tem no máximo $\log_2 n$ entradas: memória $O(\log n)$.

3. Metodologia

3.1 Ambiente de teste

Equipamento	ASUS Vivobook Go 15, código E1504FA
Processador	AMD Ryzen 5 7520U with Radeon Graphics, 2,80 GHz
Memória RAM	16,0 GB instalados (15,3 GB utilizáveis)
Sistema operacional	Windows 11, 64 bits, processador x64
Placa gráfica	AMD Radeon Graphics (integrada)
Versão do Java (JDK)	21.0.12.1

3.2 Dados gerados

- **Tipos de entrada:** *aleatória* (inteiros pseudoaleatórios, semente determinística calculada a partir de n e do número da execução, para que o experimento seja reprodutível), *ordenada* (crescente) e *invertida* (decrescente).
- **Tamanhos:** n = 100, 1.000, 5.000, 10.000 e 50.000 para os cinco algoritmos; n = 100.000 apenas para Merge e Quick. Os três algoritmos quadráticos não foram executados em 100.000 porque, no vetor aleatório, o Bubble Sort já levou cerca de 5,6 s em n = 50.000 e levaria da ordem de 20 s em 100.000, o que tornaria a bateria de testes demasiado longa sem acrescentar informação, já que a tendência quadrática já está bem estabelecida.
- **Total:** 81 combinações (algoritmo × entrada × n), todas registradas em `resultados.csv`.

3.3 Procedimento experimental

- Cada combinação é executada 5 vezes. A **primeira execução é descartada** (aquecimento do compilador JIT e das caches) e os resultados são a **média das 4 seguintes**. Por isso, algumas contagens têm casas decimais (por exemplo 4.890,75): são médias de vetores aleatórios diferentes.
- **Comparações e trocas:** contadores internos de cada algoritmo, acessados por `getComparisons()` e `getSwaps()`. "Troca" significa: Bubble, troca de vizinhos; Insertion, deslocamento (movimento); Selection, troca apenas quando o índice do menor difere do atual; Merge, escrita de volta no vetor original; Quick, troca quando i < j na partição. Como o significado difere entre algoritmos, a comparação de trocas entre algoritmos deve ser feita com cautela.
- **Tempo:** `System.nanoTime()`. O campo `tempo_ms` do CSV vem do `getExecutionTime()`, com resolução de 1 ms, e fica 0 ou arredondado para entradas pequenas. Os gráficos e tabelas deste relatório usam o campo `tempo_ns` (nanossegundos), medido pelo programa de teste, convertido para ms.
- **Memória:** bytes alocados pela *thread* durante a ordenação, obtidos de `ThreadMXBean.getThreadAllocatedBytes`.

- **Validação:** após cada ordenação, o resultado é verificado com `isSorted` e comparado com `Arrays.sort` do mesmo vetor. Todas as 81 linhas do CSV têm `ordenado = true`.
- **Análise log-log:** em escala log-log, uma função cn^k vira uma reta de inclinação k. Estimamos a inclinação por regressão linear de log(y) contra log(n) usando $n \geq 10.000$, região em que os termos de ordem menor e o aquecimento do JIT pesam menos. Para n log n, a inclinação esperada nessa faixa é ≈ 1,10 (ligeiramente acima de 1, porque o fator log n também cresce).

4. Resultados

Os dados completos estão no Apêndice A e em `resultados.csv`. A seguir, os gráficos log-log e as tabelas com os maiores tamanhos.

4.1 Comparações

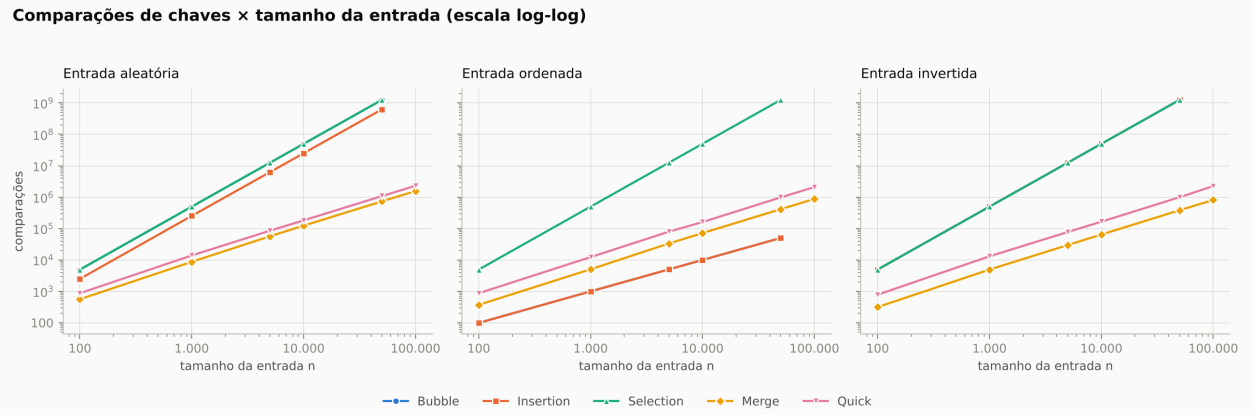


Figura 1. Comparações de chaves em função de n (log-log). Bubble e Insertion no vetor ordenado ficam sobre a reta de inclinação 1; os demais quadráticos, sobre a de inclinação 2.

4.2 Tempo de execução

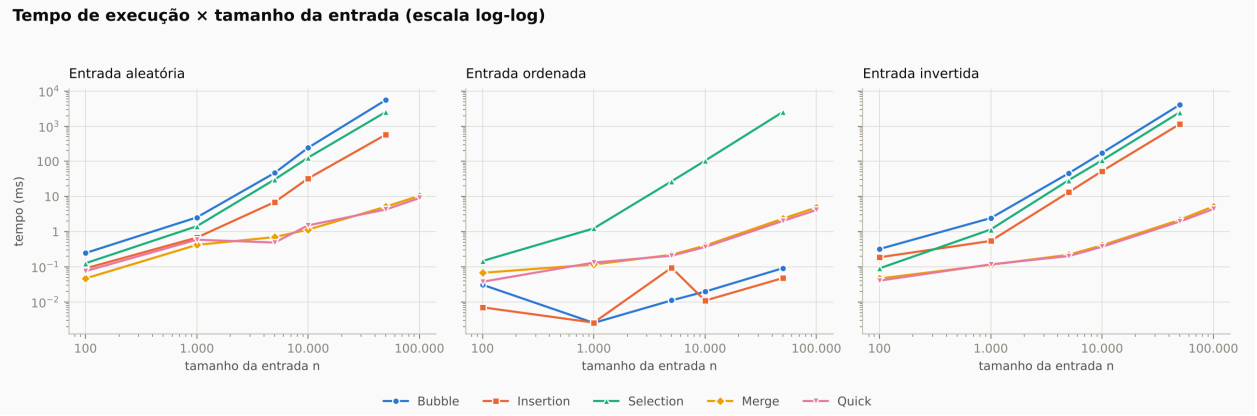


Figura 2. Tempo médio de execução (ms) em função de n (log-log). No painel central, o ponto de Bubble e Insertion em n = 1.000 e a oscilação de Insertion estão perto do limite de resolução do relógio e da influência do JIT.

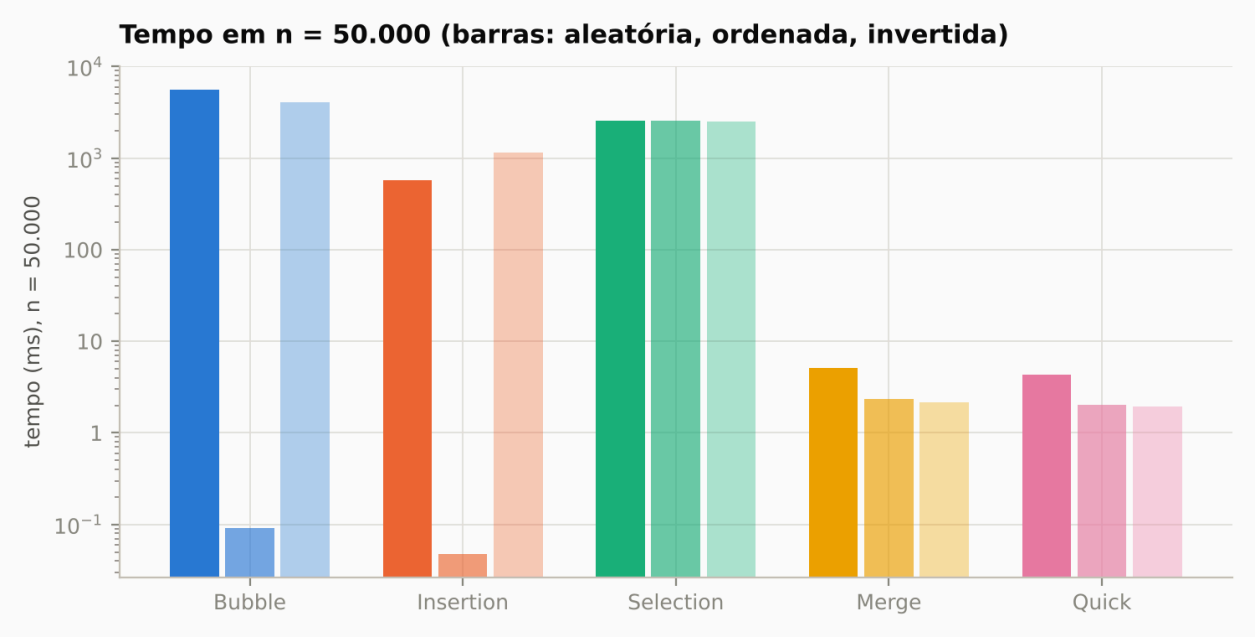


Figura 3. Tempo em n = 50.000. Eixo vertical logarítmico: a diferença entre os quadráticos e os n log n chega a três ordens de grandeza.

4.3 Trocas e movimentos

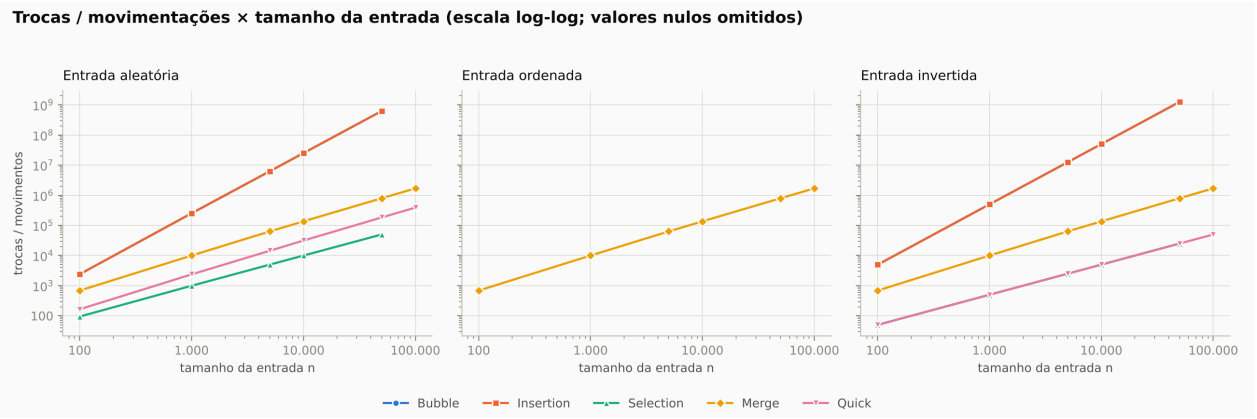


Figura 4. Trocas / movimentos em função de n (log-log). Pontos com valor zero (por exemplo Bubble e Insertion no vetor ordenado, Quick no ordenado) não existem em escala logarítmica e foram omitidos; constam das tabelas.

4.4 Memória

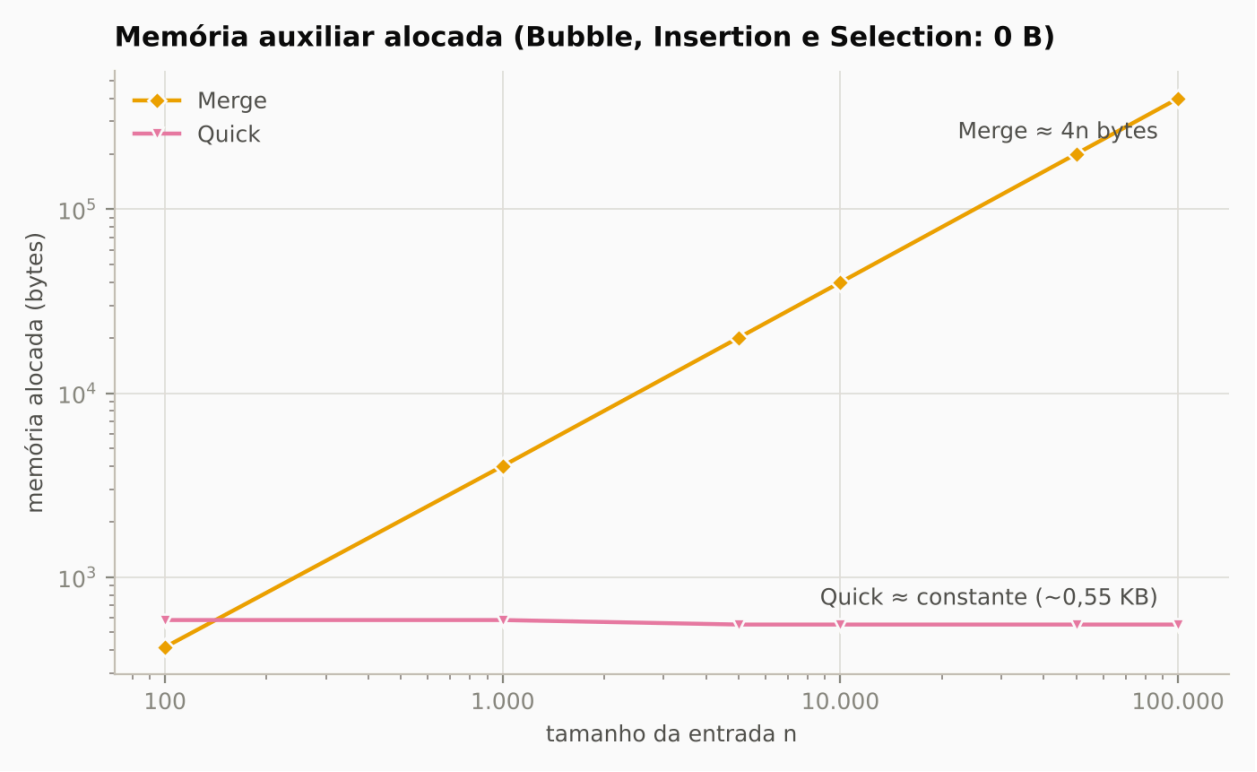


Figura 5. Memória alocada durante a ordenação. Os valores são os mesmos nos três tipos de entrada.

4.5 Tabelas

Valores em n = 50.000 (todos os algoritmos) e n = 100.000 (apenas Merge e Quick). Tempo em milissegundos.

Algoritmo	Entrada	n = 50.000			Memória (B)
		Comparações	Trocas	Tempo (ms)	
Bubble Sort	aleatória	1.249.950.380	623.562.980	5.594,31	0
	ordenada	49.999	0	0,091	0
	invertida	1.249.975.000	1.249.975.000	4.086,88	0
Insertion Sort	aleatória	623.612.969	623.562.980	577,13	0
	ordenada	49.999	0	0,048	0
	invertida	1.249.975.000	1.249.975.000	1.149,47	0
Selection Sort	aleatória	1.249.975.000	49.988	2.552,70	0
	ordenada	1.249.975.000	0	2.548,87	0
	invertida	1.249.975.000	25.000	2.497,48	0
Merge Sort	aleatória	733.203	796.496	5,13	200.016
	ordenada	413.984	796.496	2,33	200.016
	invertida	382.512	796.496	2,17	200.016

Quick Sort	aleatória	1.079.195	183.680	4,27	552
	ordenada	984.464	0	2,01	552
	invertida	996.063	25.000	1,94	552

Algoritmo	Entrada	n = 100.000			Memória (B)
		Comparações	Trocas	Tempo (ms)	
Merge Sort	aleatória	1.566.520	1.692.992	10,44	400.016
	ordenada	877.968	1.692.992	4,87	400.016
	invertida	815.024	1.692.992	5,22	400.016
Quick Sort	aleatória	2.303.360	389.494	9,00	552
	ordenada	2.096.667	0	4,12	552
	invertida	2.254.866	50.000	4,39	552

4.6 Inclinações log-log (n ≥ 10.000)

Algoritmo	Entrada	Esperada	Comparações	Tempo
Bubble Sort	aleatória	2,00	2,00	1,95
Bubble Sort	ordenada	1,00	1,00	0,95
Bubble Sort	invertida	2,00	2,00	1,96
Insertion Sort	aleatória	2,00	2,00	1,79
Insertion Sort	ordenada	1,00	1,00	0,92
Insertion Sort	invertida	2,00	2,00	1,92
Selection Sort	aleatória	2,00	2,00	1,86
Selection Sort	ordenada	2,00	2,00	1,99
Selection Sort	invertida	2,00	2,00	1,96
Merge Sort	aleatória	1,10	1,10	0,96
Merge Sort	ordenada	1,10	1,09	1,09
Merge Sort	invertida	1,10	1,10	1,10
Quick Sort	aleatória	1,10	1,10	0,75
Quick Sort	ordenada	1,10	1,12	1,05
Quick Sort	invertida	1,10	1,13	1,07

5. Discussão

5.1 O que a teoria previu e os dados confirmaram

- **Comparações seguem as fórmulas quase exatamente.** Bubble no vetor aleatório em n = 50.000 fez 1,00 vezes o valor n(n−1)/2; Insertion, 1,00 vezes $n^2/4$; Selection fez exatamente n(n−1)/2 nas três entradas. Quick no vetor aleatório com n = 100.000 ficou em 1,00 vezes 2n ln n (≈ 1,39 n $\log_2 n$), o valor esperado do Quick Sort aleatório. Merge ficou a 1,02 vezes de n $\log_2 n$ − 1,25n, e suas escritas de volta foram 1,00 vezes n⌈$\log_2 n$⌉.
- **Inclinações.** As inclinações das comparações coincidem com as esperadas com precisão de duas casas (2,00 para os quadráticos; 1,00 para Bubble e Insertion no ordenado; ≈ 1,10 para Merge e Quick). Os tempos acompanham, com desvios discutidos adiante.
- **Trocas do Selection.** No vetor aleatório, n = 50.000: observadas 49.987,75, esperadas $n - H_n \approx 49.987{,}60$; no invertido, exatamente n/2 = 25.000. O número de trocas é linear, não quadrático, o que é a característica do algoritmo.
- **Dependência da entrada.** Bubble e Insertion passam de $\Theta(n^2)$ para $\Theta(n)$ em vetor ordenado: em n = 50.000 levam 0,09 ms e 0,05 ms, contra 5.594 ms e 577 ms no aleatório. Selection mostra o contrário: o tempo no vetor ordenado é 1,00 vezes o do aleatório, ou seja, não depende da entrada. Insertion no invertido leva 1,99 vezes o tempo do aleatório, como previsto (o dobro de movimentos, $n^2/2$ contra $n^2/4$).
- **Diferença assintótica.** Em n = 50.000 no vetor aleatório, o Bubble Sort leva cerca de 1.090 vezes o tempo do Merge Sort. Em escala log-log os dois grupos (quadráticos e n log n) se afastam visivelmente à medida que n cresce.

5.2 Discrepâncias e explicações

(a) Bubble é mais rápido no vetor invertido do que no aleatório, embora tenha o dobro das trocas. No invertido o Bubble faz o mesmo número de comparações (≈ $1{,}25 \cdot 10^9$) e o dobro de trocas ($1{,}25 \cdot 10^9$ contra $6{,}2 \cdot 10^8$), mas levou 4.087 ms contra 5.594 ms, ou seja, o aleatório é 1,37 vezes mais lento. A teoria por contagem de operações não prevê isso. A explicação mais provável é a **previsão de desvios** do processador: no invertido o resultado do `if` (trocar) é sempre “sim” durante quase todo o algoritmo, e o processador acerta quase sempre; no aleatório o resultado é imprevisível e cada erro de previsão custa dezenas de ciclos. Não medimos erros de previsão diretamente, portanto a explicação permanece como hipótese coerente com o resultado.

(b) Selection é mais rápido que Bubble com as mesmas comparações. Ambos fazem ≈ $1{,}25 \cdot 10^9$ comparações no aleatório, mas o Selection leva 2.553 ms e o Bubble 5.594 ms (razão 2,19). A diferença está nas escritas em memória: o Bubble faz ≈ $6{,}2 \cdot 10^8$ trocas (cada uma lê e grava dois elementos), o Selection só ≈ $5 \cdot 10^4$. O laço interno do Selection apenas lê e compara, e o do Bubble ainda tem o desvio

imprevisível do item (a). Isso mostra que contar comparações não basta: o custo por operação também importa.

(c) Insertion é o melhor dos quadráticos. Fez metade das comparações do Bubble no aleatório ($n^2/4$ contra $n^2/2$) e o tempo reflete isso: 9,69 vezes mais rápido em n = 50.000, razão maior que 2 porque o laço do Insertion é mais simples e tem menos escritas.

(d) Inclinações de tempo abaixo do esperado em alguns casos. Para Insertion aleatório (1,79), Selection aleatório (1,86) e, sobretudo, Quick aleatório (0,75; esperado 1,10), a inclinação medida é menor que a teórica. Causas prováveis: (i) **aquecimento do JIT**: nos tamanhos pequenos e médios, parte das 4 execuções cronometradas ainda roda código interpretado ou pouco otimizado, inflando o tempo de n pequeno e achatando a curva; (ii) a faixa de regressão ($n \geq 10.000$) tem poucos pontos (2 a 3) e, para o Quick, o ponto de 10.000 (1,5 ms) ficou acima da tendência; (iii) em tempos de poucos milissegundos, ruído do sistema operacional e coleta de lixo pesam proporcionalmente muito. Para os algoritmos de ~1 ms, vale lembrar que o ponto de n = 5.000 do Quick (0,49 ms) ficou *abaixo* do de n = 1.000 (0,59 ms), o que não pode ser comportamento do algoritmo, e sim efeito de medição. Uma bateria com mais repetições e uma fase de aquecimento mais longa reduziria isso.

(e) Merge no melhor caso faz metade das comparações, mas continua $\Theta(n \log n)$. No vetor ordenado, Merge fez apenas $0{,}53 \cdot n \log_2 n$ comparações (≈ metade do aleatório), porque cada intercalação termina quando o bloco esquerdo se esgota. Já as escritas de volta são $1{,}00 \cdot n\lceil \log_2 n \rceil$ em qualquer entrada: o algoritmo não explora a ordem prévia. O tempo no ordenado é menor (2,3 ms contra 5,1 ms em n = 50.000), mas por um fator constante e por previsão de desvios, não por mudança de ordem de crescimento.

(f) Quick no ordenado e no invertido não degenerou. Com pivô fixo no meio, Quick tem a entrada "tubo de órgão" que o leva a $O(n^2)$ (observamos 10^{10} comparações e 15,8 s em um teste preliminar). Com pivô aleatório, essa entrada (e as demais) fica em $\approx 1{,}2$ a $1{,}4 \cdot n \log_2 n$ comparações. Isso confirma na prática que a robustez do Quick Sort vem da escolha do pivô, e não do algoritmo em si.

(g) Quick e Merge têm tempos quase iguais, mas por motivos diferentes. Quick faz mais comparações ($\approx 1{,}39\ n \log_2 n$ contra $\approx 0{,}94\ n \log_2 n$ do Merge no aleatório), mas muito menos movimentos ($3{,}9 \cdot 10^5$ contra $1{,}7 \cdot 10^6$ em n = 100.000) e não precisa de vetor auxiliar. O resultado são tempos muito próximos: 9,0 ms (Quick) e 10,4 ms (Merge) em n = 100.000 aleatório, com vantagem de cerca de 15% para o Quick. Nos vetores ordenado e invertido os dois ficam praticamente empatados.

(h) Memória. Merge alocou 4n + 16 bytes (um `int[n]` auxiliar: 4 bytes por inteiro mais o cabeçalho do vetor), crescendo linearmente como previsto. Quick alocou valor praticamente constante (552 B; 584 B nos dois menores tamanhos), independente de n: é a pilha fixa de 128 inteiros e o gerador aleatório. A diferença de 32 B nos menores tamanhos é provavelmente artefato da medição de alocação (primeiras execuções), não do algoritmo; não verificamos a causa. Bubble, Insertion e Selection não alocaram nada

(O(1)). Observação: o Quick Sort de pilha explícita tem memória O(log n) no máximo e, com a pilha de 128 posições, isso nunca é um limite na prática, mas o número 552 é uma constante da implementação, não uma medição de crescimento.

(i) Resolução do relógio. `getExecutionTime()` em ms (resolução de 1 ms) devolve 0 para tudo abaixo de ≈ 1 ms, o que torna o campo `tempo_ms` inadequado para n pequeno. Por isso todas as análises usam `tempo_ns`. Em n ≥ 50.000, ambos concordam dentro do erro de arredondamento.

5.3 Trade-offs

Critério	Melhor escolha	Observação
Vetores grandes e desempenho médio	Quick Sort (pivô aleatório)	Menor tempo e memória quase nula; não é estável e tem pior caso $O(n^2)$ improvável.
Garantia de pior caso e estabilidade	Merge Sort	$\Theta(n \log n)$ sempre e estável, ao custo de $O(n)$ de memória extra.
Vetores pequenos ou quase ordenados	Insertion Sort	$\Theta(n)$ no ordenado e o menor custo constante entre os quadráticos.
Poucas escritas em memória	Selection Sort	No máximo $n-1$ trocas, mas sempre $\Theta(n^2)$ comparações.
Fins didáticos	Bubble Sort	Pior desempenho em todos os casos medidos, exceto no vetor ordenado.

6. Conclusão

6.1 Aprendizados

- As contagens de comparações e de movimentos confirmaram as fórmulas teóricas com grande precisão, e as inclinações log-log das comparações reproduziram exatamente 1, 2 e ≈ 1,1.
- Os tempos seguem a mesma ordem de crescimento, mas com ruído e efeitos de hardware (previsão de desvios, JIT, resolução do relógio) que a análise assintótica deliberadamente ignora. Esses efeitos explicam diferenças de fator constante, nunca de ordem de crescimento.
- Em n = 50.000 a diferença entre um algoritmo $\Theta(n^2)$ e um $\Theta(n \log n)$ é de três ordens de grandeza (≈ 1.090× entre Bubble e Merge).
- A escolha do pivô determina o comportamento prático do Quick Sort: com pivô fixo no meio, uma entrada específica o tornou quadrático; com pivô aleatório, não.

6.2 Recomendações

- Para uso geral em vetores grandes de inteiros: Quick Sort com pivô aleatório (ou Merge Sort, se estabilidade ou garantia de pior caso forem exigidas).
- Para vetores pequenos ou quase ordenados: Insertion Sort, e é a razão pela qual implementações reais (como as bibliotecas padrão) o usam em trechos pequenos dentro de algoritmos híbridos.
- Bubble e Selection Sort não são recomendados em aplicações reais.

6.3 Trabalhos futuros

- Aumentar o número de repetições e usar uma fase de aquecimento mais longa (ou uma ferramenta como JMH) para reduzir o ruído dos tempos pequenos.
- Incluir entradas com muitos valores repetidos e quase ordenadas, e variantes como Quick Sort com mediana de três e Insertion Sort em trechos pequenos.
- Medir erros de previsão de desvios e falhas de cache (por exemplo com contadores de hardware) para confirmar a explicação do item (a).
- Comparar com `Arrays.sort` (Dual-Pivot Quicksort) e com ordenações paralelas.

Referências

- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. MIT Press, 2009.
- SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.
- KNUTH, D. E. *The Art of Computer Programming, vol. 3: Sorting and Searching*. 2. ed. Addison-Wesley, 1998.
- SOUZA, A. B. de. Enunciado do Projeto Final: Batalha dos Algoritmos de Ordenação. Projeto e Análise de Algoritmos, 2026.

Apêndice A. Dados completos

Cópia integral de `resultados.csv`. Tempo em ms (a partir de `tempo_ns`).

Algoritmo	Entrada	n	Comparações	Trocas	Tempo (ms)	Memória (B)
Bubble Sort	aleatória	100	4.890,75	2.402,00	0,246	0
Insertion Sort	aleatória	100	2.498,25	2.402,00	0,090	0
Selection Sort	aleatória	100	4.950,00	94,50	0,126	0
Merge Sort	aleatória	100	560,75	688,00	0,046	416
Quick Sort	aleatória	100	872,25	165,00	0,075	584
Bubble Sort	ordenada	100	99,00	0,00	0,031	0
Insertion Sort	ordenada	100	99,00	0,00	0,007	0
Selection Sort	ordenada	100	4.950,00	0,00	0,146	0
Merge Sort	ordenada	100	372,00	688,00	0,067	416
Quick Sort	ordenada	100	873,00	0,00	0,037	584
Bubble Sort	invertida	100	4.950,00	4.950,00	0,319	0
Insertion Sort	invertida	100	4.950,00	4.950,00	0,186	0
Selection Sort	invertida	100	4.950,00	50,00	0,090	0
Merge Sort	invertida	100	316,00	688,00	0,046	416
Quick Sort	invertida	100	783,00	50,00	0,040	584
Bubble Sort	aleatória	1.000	498.516,50	252.349,75	2,54	0
Insertion Sort	aleatória	1.000	253.344,00	252.349,75	0,675	0
Selection Sort	aleatória	1.000	499.500,00	993,75	1,42	0
Merge Sort	aleatória	1.000	8.719,25	9.984,00	0,419	4.016
Quick Sort	aleatória	1.000	13.923,00	2.369,25	0,587	584
Bubble Sort	ordenada	1.000	999,00	0,00	0,003	0
Insertion Sort	ordenada	1.000	999,00	0,00	0,003	0
Selection Sort	ordenada	1.000	499.500,00	0,00	1,23	0
Merge Sort	ordenada	1.000	5.052,00	9.984,00	0,116	4.016
Quick Sort	ordenada	1.000	12.343,00	0,00	0,132	584
Bubble Sort	invertida	1.000	499.500,00	499.500,00	2,43	0
Insertion Sort	invertida	1.000	499.500,00	499.500,00	0,543	0
Selection Sort	invertida	1.000	499.500,00	500,00	1,15	0
Merge Sort	invertida	1.000	4.932,00	9.984,00	0,114	4.016
Quick Sort	invertida	1.000	13.082,00	500,00	0,117	584
Bubble Sort	aleatória	5.000	12.491.404,75	6.273.235,50	46,70	0
Insertion Sort	aleatória	5.000	6.278.226,50	6.273.235,50	6,92	0
Selection Sort	aleatória	5.000	12.497.500,00	4.990,50	30,36	0
Merge Sort	aleatória	5.000	56.784,50	63.160,00	0,698	20.016
Quick Sort	aleatória	5.000	85.377,25	14.426,50	0,489	552
Bubble Sort	ordenada	5.000	4.999,00	0,00	0,011	0
Insertion Sort	ordenada	5.000	4.999,00	0,00	0,094	0
Selection Sort	ordenada	5.000	12.497.500,00	0,00	26,99	0
Merge Sort	ordenada	5.000	33.356,00	63.160,00	0,219	20.016
Quick Sort	ordenada	5.000	79.454,00	0,00	0,206	552
Bubble Sort	invertida	5.000	12.497.500,00	12.497.500,00	45,49	0
Insertion Sort	invertida	5.000	12.497.500,00	12.497.500,00	13,16	0
Selection Sort	invertida	5.000	12.497.500,00	2.500,00	28,95	0

Merge Sort	invertida	5.000	29.804,00	63.160,00	0,219	20.016
Quick Sort	invertida	5.000	77.361,00	2.500,00	0,201	552
Bubble Sort	aleatória	10.000	49.979.117,75	25.053.709,25	243,59	0
Insertion Sort	aleatória	10.000	25.063.699,75	25.053.709,25	32,50	0
Selection Sort	aleatória	10.000	49.995.000,00	9.989,25	127,68	0
Merge Sort	aleatória	10.000	123.660,25	136.320,00	1,14	40.016
Quick Sort	aleatória	10.000	182.746,25	31.392,75	1,50	552
Bubble Sort	ordenada	10.000	9.999,00	0,00	0,020	0
Insertion Sort	ordenada	10.000	9.999,00	0,00	0,011	0
Selection Sort	ordenada	10.000	49.995.000,00	0,00	104,21	0
Merge Sort	ordenada	10.000	71.712,00	136.320,00	0,399	40.016
Quick Sort	ordenada	10.000	161.000,00	0,00	0,365	552
Bubble Sort	invertida	10.000	49.995.000,00	49.995.000,00	173,54	0
Insertion Sort	invertida	10.000	49.995.000,00	49.995.000,00	52,56	0
Selection Sort	invertida	10.000	49.995.000,00	5.000,00	106,69	0
Merge Sort	invertida	10.000	64.608,00	136.320,00	0,407	40.016
Quick Sort	invertida	10.000	165.605,00	5.000,00	0,370	552
Bubble Sort	aleatória	50.000	1.249.950.379,75	623.562.979,50	5.594,31	0
Insertion Sort	aleatória	50.000	623.612.969,25	623.562.979,50	577,13	0
Selection Sort	aleatória	50.000	1.249.975.000,00	49.987,75	2.552,70	0
Merge Sort	aleatória	50.000	733.203,00	796.496,00	5,13	200.016
Quick Sort	aleatória	50.000	1.079.195,00	183.680,50	4,27	552
Bubble Sort	ordenada	50.000	49.999,00	0,00	0,091	0
Insertion Sort	ordenada	50.000	49.999,00	0,00	0,048	0
Selection Sort	ordenada	50.000	1.249.975.000,00	0,00	2.548,87	0
Merge Sort	ordenada	50.000	413.984,00	796.496,00	2,33	200.016
Quick Sort	ordenada	50.000	984.464,00	0,00	2,01	552
Bubble Sort	invertida	50.000	1.249.975.000,00	1.249.975.000,00	4.086,88	0
Insertion Sort	invertida	50.000	1.249.975.000,00	1.249.975.000,00	1.149,47	0
Selection Sort	invertida	50.000	1.249.975.000,00	25.000,00	2.497,48	0
Merge Sort	invertida	50.000	382.512,00	796.496,00	2,17	200.016
Quick Sort	invertida	50.000	996.063,00	25.000,00	1,94	552
Merge Sort	aleatória	100.000	1.566.520,25	1.692.992,00	10,44	400.016
Quick Sort	aleatória	100.000	2.303.360,50	389.494,50	9,00	552
Merge Sort	ordenada	100.000	877.968,00	1.692.992,00	4,87	400.016
Quick Sort	ordenada	100.000	2.096.667,00	0,00	4,12	552
Merge Sort	invertida	100.000	815.024,00	1.692.992,00	5,22	400.016
Quick Sort	invertida	100.000	2.254.866,00	50.000,00	4,39	552